

DEPARTMENT OF COMPUTING

PRISON MANAGEMENT SYSTEM

Entity Relationships Document

Item	Detail
Course	Software Engineering, Group Mini-Project
Group	Group aa2
Supervisor	Chisomo Sayah
Prepared by	Herbert Mateyu
Date	28th September 2026
Related documents	Functional Requirements Document; Key Entities document

Pillar 1: Purpose of This Document

This document explains how the different parts of the Prison Management System are connected to one another. The system stores a lot of information about people, places, legal matters and daily events in a prison, and that information is only useful if it is connected correctly; for example, it is not enough to store a list of inmates and a separate list of cells, since the system must also know which inmate is in which cell, since when, and who made the decision. These connections are called relationships, and they are the subject of this document.

The Key Entities document defines twenty-four entities, including the six that describe the inmate health module, and lists the primary and foreign keys that connect them. Working through that table gives thirty-four formal relationships, each enforced by a foreign key, five many-to-many links built out of pairs of those relationships, and four further links that the application keeps track of without a real foreign key, called soft references. This document explains all of them, one at a time, in the same order as the Key Entities document, and finishes with a single summary table for quick reference. It focuses on how the entities are related, not on how data moves through the system step by step.

Pillar 2: Key Terms

A few terms are used throughout this document. An *entity* is a type of thing the system keeps information about, such as Inmate or Cell, and becomes a table in the database; an *attribute* is one piece of information about an entity, such as an inmate's `date_of_birth`, and becomes a column. A *primary key* is the attribute that uniquely identifies each record in an entity, while a *foreign key* is an attribute in one entity that stores the primary key of a related entity, which is what actually creates a relationship. In any relationship, the entity whose primary key is being pointed to is called the *parent*, and the entity holding the foreign key is the *child*; the rule to remember is that the foreign key always lives on the child, or "many", side.

A relationship's *cardinality* is its counting rule. Every relationship in this system is *one-to-many*, meaning one record on the parent side can relate to many records on the child side, while each child record relates to only one parent. Some real-world connections are *many-to-many*, where many records on each side relate to many on the other; a database cannot store this directly, so it is split into two one-to-many relationships through a *junction table*, whose own primary key is often a *composite key* made of the two foreign keys together. A *soft reference* is different again: it is a link the application keeps track of, but that the database does not enforce with a real foreign key, usually because one column has to point at more than one table.

Pillar 3: The Big Picture

Two patterns stand out once all thirty-four relationships are laid side by side. First, every one of them is one-to-many, which is normal for a well-designed database, since a many-to-many link is always split into two one-to-many relationships and there is no need in this system for a one-to-one link. Second, the entities are not equally connected: *User* is now the most connected entity, appearing in fourteen of the thirty-four relationships, because almost every action in the system, including every step of a health consultation, has to be traced back to a member of staff. *Inmate* follows with seven relationships and still sits at the centre of custody, legal, discipline, visitor, rehabilitation and now health information, which matches its description in the Key Entities document as the central entity of the system.

The easiest way to see how everything fits together is to follow one inmate. When the inmate is admitted, a record is created in *Inmate*, and the inmate is placed in a cell, which creates a *CellAssignment* linking the inmate, the cell and the staff member who made the decision. At the same time, the Medical Officer opens a *HealthRecord* during the enrollment screening, and from then on every consultation, known condition, prescribed medication and its administration, and every scheduled appointment is attached to that health record or directly to the inmate, each one naming the staff member responsible.

In parallel, a Case Officer records the legal case for the inmate, attaches the offences involved through *CaseCrime*, and records the sentence that follows; the system uses the sentence's release date to send a notification to staff as release approaches. During the inmate's stay, everyday events add more records around the same inmate: a disciplinary incident is linked to the inmate, each family visit creates a visit record linked to a visitor, and joining a rehabilitation programme creates an enrollment with an attendance record for every session attended. Finally, every creation, change or deletion along the way is written to the audit log with the name of the staff member responsible.

Pillar 4: The Thirty-Four Formal Relationships

Each relationship below is presented as one paragraph naming the two entities, the exact foreign key that creates the link, the functional requirement it supports, and a worked example, grouped by business area in the same order as the Key Entities document.

4.1 Access and Administration

R1. User and Role

A role is a job description that the system understands, such as Records Officer, Case Officer, Medical Officer or Super Admin. Every staff member who has an account is given exactly one role, and many staff members can share the same role, which is why *Role* is the parent table and *User* is the child: the *User* table carries a foreign key called *role_id* that points to the primary key *role_id* in *Role*. Because the foreign key sits in *User*, one *Role* row can be pointed to by many *User* rows, but each *User* row points to only one *Role*. This is what lets the Super Admin change someone's role (FR-002) by updating a single foreign key value, and it is the foundation of role-based access control (FR-004), since the system reads a user's role and then that role's permissions before allowing any action. For example, three staff members who are all Records Officers share one *Role* row, so a change to what Records Officers may do is made once and applies to all three of them immediately.

4.2 Inmate and Custody

R2. Inmate and CellAssignment

The system does not simply write a cell number on an inmate's file; each time an inmate is placed in a cell, a separate *CellAssignment* record is created, so *CellAssignment* is the child of *Inmate* and its foreign key *inmate_id* refers to the primary key *inmate_id* in *Inmate*. One inmate can therefore build up many assignment records over a stay, one for every cell they have lived in, while each individual assignment belongs to one inmate only. Keeping assignments as their own records, instead of storing a single *cell_id* on the *Inmate* table, is what preserves the transfer history required by FR-009, which asks that staff be able to reassign an inmate to a different cell; when a move happens, the old assignment is closed by filling in its *date_released* and a new one is opened with a fresh *date_assigned*. For example, inmate 1042 is admitted on 3 March and placed in Cell B-12, then moved to Cell C-04 on 10 August; the system now holds two assignment records for that inmate, the first closed and the second still open, so anyone reviewing the file can see exactly where the inmate has been housed and when.

R3. Cell and CellAssignment

A cell is a room that, over time, houses many inmates, and at any one moment may hold several of them up to its capacity, so *Cell* is the parent and *CellAssignment* the child through the foreign key *cell_id*, which refers to the primary key *cell_id* in *Cell*. The *Cell* entity stores its capacity and current_occupancy, and comparing these two figures before creating a new assignment is how the system enforces FR-008, refusing a placement once a cell is full; the classification rules in FR-007 depend on the same link, since an inmate's classification is checked against the cell's *classification_type* before the assignment is allowed. For example, Cell B-12 has a capacity of six and

already has six open assignment records, so a seventh placement is rejected until one inmate is transferred out and that assignment is closed.

R4. User and CellAssignment (assigned_by)

The system also remembers which staff member made each placement: every CellAssignment record names the user who created it through the foreign key assigned_by, which refers to the primary key user_id in User, and one user can make many assignments over time. Naming the column assigned_by rather than a plain user_id makes clear what role this user plays in the record, a pattern repeated elsewhere in the design as recorded_by, reported_by and logged_by, and it gives the accountability that FR-007 and FR-009 rely on when a placement or transfer is questioned. For example, when Records Officer Grace places an inmate in a cell, the new assignment carries her user_id in assigned_by, so if management later needs to know who authorised the placement, the answer is one lookup away.

4.3 Health Management

R5. Inmate and HealthRecord

Every inmate's medical information is gathered into a HealthRecord, opened at admission and updated for as long as the inmate remains in custody, so HealthRecord is the child of Inmate through the foreign key inmate_id, which refers to the primary key inmate_id in Inmate. This satisfies FR-029, which requires that a health record be created for an inmate together with a screening at enrollment, and FR-034, which requires that an inmate's full health history be viewable in one place. For example, when inmate 1042 is admitted, the Medical Officer's screening creates a single HealthRecord row carrying that inmate's blood group and initial screening results, and it is the same row that every later consultation, condition and appointment will attach to.

R6. User and HealthRecord (created_by)

The health record still belongs to the inmate, as established in Relationship 5 above; this second link only records which Medical Officer opened it, through the foreign key created_by, which refers to the primary key user_id in User. One Medical Officer can open many health records over time while each record names one creator, giving FR-029's screening step the same staff accountability used throughout the system. For example, Medical Officer Chisale performs the enrollment screening for a new inmate, and the resulting HealthRecord still belongs to that inmate but additionally carries her user_id in created_by, showing who carried out the screening.

R7. HealthRecord and MedicalCondition

Because an inmate can have more than one known condition or allergy, each one is stored as its own row in MedicalCondition rather than crammed into the health record itself, so MedicalCondition is the child of HealthRecord through the foreign key health_record_id, which refers to the primary key health_record_id in HealthRecord. One health record can therefore carry many condition rows, satisfying FR-030's requirement that known conditions and allergies be recorded and kept up to date, while each condition row belongs to one health record only. For example, an inmate's file might show two separate MedicalCondition rows, one for a peanut allergy and one for asthma, both pointing at the same HealthRecord.

R8. User and MedicalCondition (recorded_by)

Each condition also names the staff member who recorded it, through the foreign key `recorded_by`, which refers to the primary key `user_id` in `User`, so one officer can record many conditions while each condition names one recorder. For example, the Medical Officer who notes a newly discovered allergy is stored as `recorded_by` on that particular `MedicalCondition` row.

R9. HealthRecord and Consultation

A Consultation is a single visit to the Medical Officer, complete with its own complaint, diagnosis and treatment, and because an inmate may be seen many times, Consultation is the child of HealthRecord through the foreign key `health_record_id`, referring to the primary key `health_record_id` in `HealthRecord`. This is how FR-031 is satisfied, since it requires that consultations, including diagnosis and treatment, be recorded against the correct inmate; one health record can carry many consultations, while each consultation belongs to one health record. For example, an inmate seen twice for the same chest complaint ends up with two Consultation rows, both attached to the same HealthRecord.

R10. User and Consultation (consulted_by)

Every consultation also names the Medical Officer who conducted it, through the foreign key `consulted_by`, which refers to the primary key `user_id` in `User`, so one officer can conduct many consultations while each consultation names one officer. For example, when Dr Phiri examines an inmate, her `user_id` is stored as `consulted_by` on that consultation.

R11. Consultation and Medication

When a consultation results in a prescription, that prescription is stored as its own Medication row rather than as text inside the consultation, so Medication is the child of Consultation through the foreign key `consultation_id`, which refers to the primary key `consultation_id` in `Consultation`. One consultation can therefore lead to several medications at once, which is how FR-032 supports recording medication prescribed to an inmate, while each medication traces back to the single consultation that prescribed it. For example, a consultation for a chest infection might result in two separate Medication rows, an antibiotic and a pain reliever, both linked to that one consultation.

R12. User and Medication (prescribed_by)

Each medication also names the prescribing officer, through the foreign key `prescribed_by`, which refers to the primary key `user_id` in `User`, so one officer can prescribe many medications while each medication has one prescriber. For example, the officer who prescribes the antibiotic is recorded as `prescribed_by` on that Medication row.

R13. Medication and MedicationAdministration

Because a single prescription is usually given more than once, each individual dose is recorded separately in MedicationAdministration, which is the child of Medication through the foreign key `medication_id`, referring to the primary key `medication_id` in `Medication`. This is how FR-032's requirement to record medication administration is met in practice: one Medication row can have many administration records, one for every dose actually given, while each administration record

belongs to one medication. For example, a course taken twice a day for five days produces ten MedicationAdministration rows against the single Medication row for that course.

R14. User and MedicationAdministration (administered_by)

Each dose given also names the staff member who gave it, through the foreign key administered_by, which refers to the primary key user_id in User, so one nurse or officer can administer many doses while each dose names one administerer. For example, the nurse who gives an inmate's 8am dose is recorded as administered_by for that particular administration.

R15. HealthRecord and MedicalAppointment

Scheduled appointments and referrals to outside facilities are stored in MedicalAppointment, which is the child of HealthRecord through the foreign key health_record_id, referring to the primary key health_record_id in HealthRecord. One health record can carry many appointments over time, which satisfies FR-033's requirement to schedule and track medical appointments including external referrals, while each appointment belongs to one health record. For example, an inmate referred to an outside clinic and later booked for a follow-up ends up with two MedicalAppointment rows on the same HealthRecord.

R16. User and MedicalAppointment (scheduled_by)

Every appointment also names the staff member who booked it, through the foreign key scheduled_by, which refers to the primary key user_id in User, so one officer can schedule many appointments while each appointment names one scheduler. For example, the officer who books an external referral is recorded as scheduled_by on that appointment, which is also the row that FR-036's reminder notifications later point back to.

4.4 Legal and Sentencing

R17. Inmate and Case

A case is the legal file about a charge or a court matter, and Case is the child of Inmate through the foreign key inmate_id, which refers to the primary key inmate_id in Inmate; an inmate can be connected to more than one case over time, for example a new charge while already serving a sentence, or a case that was later appealed, but each individual case belongs to one inmate only. This satisfies FR-013, which requires that a case record be linked to the correct inmate, and it is also how FR-015 works, since a change to a case's status between ongoing, closed and appealed is stored on the Case row and shows up on the inmate's profile because the profile retrieves every case carrying that inmate's inmate_id. For example, inmate 1042 has an ongoing case, CR-2026/118, and an earlier closed case, CR-2021/044, and both rows carry inmate_id 1042, so opening the inmate's profile shows both.

R18. User and Case (recorded_by)

Every case also names the staff member, typically a Case Officer, who recorded it, through the foreign key recorded_by, which refers to the primary key user_id in User, and one officer can record many cases while each case names one recorder, which supports the accountability behind FR-013. This complements the audit log described later: the audit log records every change in a general way, while

recorded_by keeps a permanent, easily visible note of who first created the case. For example, when Case Officer Peter records a new case, the case row shows his user_id in recorded_by, so anyone opening the case can see who captured it.

R19. Case and CaseCrime

One case can involve several offences at once, so instead of squeezing them into one field, the system creates one small CaseCrime record for every offence attached to a case; CaseCrime is the child of Case through the foreign key case_id, referring to the primary key case_id in Case. CaseCrime is a junction table, whose only job is to connect a case to a crime, and together with the next relationship it forms the many-to-many link between cases and crimes described in FR-013; its own primary key is a composite of case_id and crime_id, which stops the same offence being attached to the same case twice. For example, a case involving both theft and assault produces two CaseCrime rows, one linking it to theft and one to assault.

R20. Crime and CaseCrime

Crime is a reference list of offences kept in one place, and the same offence can appear in many different cases, so Crime is the parent and CaseCrime the child through the foreign key crime_id, which refers to the primary key crime_id in Crime. Storing offences once, instead of typing an offence name into every case, avoids duplication and spelling inconsistencies, and it is what FR-013 relies on when it associates crimes with a case. For example, the offence Theft exists as a single row in Crime and can be linked to two hundred different cases through two hundred CaseCrime rows, so correcting its description is done in one place only.

R21. Case and Sentence

A sentence is the penalty that comes out of a case, and Sentence is the child of Case through the foreign key case_id, referring to the primary key case_id in Case; a case can result in one or more sentences, for example where a court imposes a separate sentence for each offence, but each sentence belongs to one case only. This is what FR-019 requires when it says a sentence must be linked to the correct case, and the Sentence table also holds the duration and start_date entered by the Case Officer, the release_date the system calculates automatically under FR-020, and the status of active, completed or commuted required by FR-021. For example, a case ending with a twenty-four-month sentence starting on 1 June 2026 has the system calculate a release date of 1 June 2028 and store it on that same Sentence row.

R22. Inmate and Sentence

A sentence is also connected directly to the inmate who must serve it, so Sentence carries a second foreign key, inmate_id, referring to the primary key inmate_id in Inmate; one inmate can have many sentences over time, and each sentence belongs to one inmate. Since a Case already belongs to one Inmate (Relationship 17), the inmate on a sentence could also be reached by going through the case, so this direct link is really a shortcut for faster searching. For example, to find every sentence for inmate 1042, the system can search the Sentence table directly for inmate_id 1042, without first going through that inmate's cases.

4.5 Discipline

R23. User and Incident (reported_by)

Every disciplinary incident records which staff member reported it, through the foreign key reported_by, which refers to the primary key user_id in User; one staff member, usually a Disciplinary Officer, can report many incidents, but each incident has one reporter, which supports FR-010's requirement that an incident be recorded with its details, and later helps if an incident is questioned. For example, when Disciplinary Officer Alinafe logs a fight in the exercise yard, the incident row carries her user_id in reported_by.

R24. Incident and IncidentInmate

One incident can involve several inmates, such as a fight with two people fighting and three watching, so the system creates one IncidentInmate record for every inmate involved; IncidentInmate is the child of Incident through the foreign key incident_id, referring to the primary key incident_id in Incident, and it is a junction table whose own primary key is the composite of incident_id and inmate_id. This is how FR-010 supports an incident involving one or more inmates, and once the incident is logged, recording its outcome under FR-011 is done once, on the single Incident row. For example, a fight involving five inmates produces five IncidentInmate rows that all share the same incident_id.

R25. Inmate and IncidentInmate

Over time, one inmate can be involved in many incidents, and each IncidentInmate record points to exactly one inmate through the foreign key inmate_id, referring to the primary key inmate_id in Inmate; this is what makes it possible to satisfy FR-012 and view an inmate's full disciplinary history, by finding every IncidentInmate row for that inmate and then reading the linked incidents. The extra column role_in_incident, which records whether the inmate was a perpetrator, witness or victim, is stored on this link because it describes the inmate's part in one particular incident, not a fact about the inmate or the incident alone. For example, inmate 1042 was a witness in one incident in March and a perpetrator in another in July, so two IncidentInmate rows carry that inmate's id with different role_in_incident values, and both incidents appear in the disciplinary history.

4.6 Visitors

R26. Visitor and Visit

A visitor is a person whose identity is recorded once, and the same person can come back many times, with a new Visit record created every time they come; Visit is the child of Visitor through the foreign key visitor_id, referring to the primary key visitor_id in Visitor. Separating the person from the event this way avoids retyping a visitor's name, ID number and phone number on every visit, which is part of how FR-016 and FR-017 are satisfied, and it lets a returning visitor be recognised while their relationship_to_inmate is stored once with the person. For example, a mother who visits her son every month has one Visitor row and, after a year, twelve Visit rows all carrying that same visitor_id.

R27. Inmate and Visit

Every visit is a visit to one particular inmate, so Visit also carries the foreign key inmate_id, referring to the primary key inmate_id in Inmate, and one inmate can receive many visits over time. This link is

what lets FR-018 be satisfied, since staff can view an inmate's visitor history by retrieving every Visit row carrying that inmate's inmate_id; together with the previous relationship, Visit therefore acts as the link between Visitor and Inmate, but as a full entity with its own visit_id, visit_datetime and duration, because the same visitor can visit the same inmate on many different occasions. For example, asking for the visitor history of inmate 1042 returns every visit with inmate_id 1042, each showing who came, when and for how long.

R28. User and Visit (logged_by)

The system also records which staff member, usually a Visitor Officer, logged each visit, through the foreign key logged_by, referring to the primary key user_id in User, following the same accountability pattern as assigned_by, recorded_by and reported_by elsewhere in the design. For example, when Visitor Officer Chikondi logs an afternoon visit, the Visit row carries her user_id in logged_by.

4.7 Rehabilitation

R29. Inmate and Enrollment

An enrollment records that an inmate has joined a rehabilitation programme, and Enrollment is the child of Inmate through the foreign key inmate_id, referring to the primary key inmate_id in Inmate; an inmate can join many programmes over time, for example carpentry and later counselling, while each enrollment belongs to one inmate. This is how FR-024's requirement for a rehabilitation history is satisfied, by retrieving every Enrollment row for that inmate together with its attendance records. For example, inmate 1042 is enrolled in a carpentry course in February and a counselling programme in September, giving two Enrollment rows for that inmate.

R30. RehabProgram and Enrollment

A rehabilitation programme such as vocational training or counselling is offered once but attended by many inmates, so RehabProgram is the parent and Enrollment the child through the foreign key program_id, referring to the primary key program_id in RehabProgram, satisfying FR-022's requirement to record enrollment in a programme. Together with the previous relationship, Enrollment is the link between inmates and programmes, but unlike a plain junction table it carries its own enrollment_id, enrollment_date and status, because it also has attendance records attached to it. For example, the carpentry programme has thirty enrolled inmates, so thirty Enrollment rows carry its program_id.

R31. Enrollment and AttendanceRecord

Attendance is recorded session by session, so AttendanceRecord is the child of Enrollment through the foreign key enrollment_id, referring to the primary key enrollment_id in Enrollment; one enrollment collects many attendance records over time, one for each session held, while each attendance record belongs to one enrollment. Attendance is attached to the enrollment rather than directly to the inmate or the programme because it is a fact about one particular inmate in one particular programme, and each row stores the session date, a present flag and any remarks required by FR-023. For example, a carpentry course that meets three times a week gives one inmate's enrollment around twelve attendance rows a month, some marked present and some absent.

R32. User and AttendanceRecord (recorded_by)

Each attendance record also names the staff member, typically a Rehabilitation Officer, who took the register, through the foreign key `recorded_by`, referring to the primary key `user_id` in `User`, following the same accountability pattern used elsewhere. For example, when Rehabilitation Officer Tiyamike marks Monday's counselling register, each row she creates carries her `user_id` in `recorded_by`.

4.8 System Support**R33. User and AuditLog**

The audit log is the system's diary: every time someone creates, changes or deletes something, an entry is written naming the person responsible, so `AuditLog` is the child of `User` through the foreign key `user_id`, referring to the primary key `user_id` in `User`, and one user can be responsible for many entries while each entry names one user. This is what lets the Super Admin filter the log by user under FR-026, and it satisfies FR-025's requirement that each entry record the action, the timestamp and the responsible staff member; each entry also stores the `action_type`, the `entity_name`, the `record_id`, and the old and new values, described further under the soft references below. For example, when a Records Officer corrects an inmate's date of birth, the audit log gains an entry with the officer's `user_id`, the action `Update`, the entity `Inmate`, that inmate's record id, and the old and new dates.

R34. User and Notification (recipient)

A notification is a message the system sends to a staff member, for example to warn that a sentence is nearly complete, that a new incident has been logged, or that a medical appointment is coming up, so `Notification` carries the foreign key `recipient_user_id`, referring to the primary key `user_id` in `User`; one user can receive many notifications, and each notification is addressed to one user. The column is named `recipient_user_id` rather than a plain `user_id` to make clear that this user is the receiver of the message rather than its author, since notifications are created by the system itself, and the `is_read` column lets the system tell which notifications a user has already seen. For example, seven days before an inmate's release date the system sends a notification to the relevant Case Officer, with that officer's `user_id` in `recipient_user_id` and `is_read` set to false until it is opened.

Pillar 5: The Five Many-to-Many Links

Some real-world connections cannot be stored directly in a database, and when many records on one side can connect to many records on the other, the connection is split into two ordinary one-to-many relationships through a junction table in the middle. The five many-to-many links in this system are described below; each one adds no new relationships to the count of thirty-four, since it is made up of relationships already described in Pillar 4.

Inmate and Cell, through CellAssignment. Over time, one inmate can live in many cells, and one cell can house many inmates. CellAssignment holds one record for every placement, made up of Relationships 2 and 3, and because it carries its own date_assigned, date_released and assigned_by, it also acts as the transfer history required by FR-009; its own primary key is assignment_id.

Case and Crime, through CaseCrime. One case can involve many offences, and one offence can appear in many cases. CaseCrime holds one record for each pairing, made up of Relationships 19 and 20, and being a pure junction table it holds nothing but the two foreign keys, case_id and crime_id, which together form its composite primary key.

Incident and Inmate, through IncidentInmate. One incident can involve many inmates, and one inmate can be involved in many incidents. IncidentInmate holds one record per inmate per incident, made up of Relationships 24 and 25, with a composite primary key of incident_id and inmate_id and one extra attribute, role_in_incident.

Inmate and RehabProgram, through Enrollment. One inmate can join many rehabilitation programmes, and one programme can be joined by many inmates. Enrollment sits in the middle, made up of Relationships 29 and 30, and is a full entity with its own enrollment_id, since it also carries attendance records through Relationship 31.

Visitor and Inmate, through Visit. One visitor can visit many inmates, and one inmate can be visited by many visitors. Visit sits in the middle, made up of Relationships 26 and 27, and is a full entity with its own visit_id, visit_datetime and duration, plus a third foreign key, logged_by, described in Relationship 28.

Splitting a many-to-many link through a junction table is what allows the system to keep history, as with transfers, to store extra facts about each pairing, as with the role in an incident, and to avoid repeating information, as with offences. A design that tried to store these links inside a single column would quickly become inconsistent and hard to search.

Pillar 6: The Four Soft References

Most links in the system are protected by the database itself through foreign keys. A small number of links, however, use a single column to point at records in more than one table, which an ordinary foreign key cannot do; these are kept track of by the application code instead, and are called soft references.

Notification and Sentence. When the system warns staff that a sentence is nearing completion, the notification needs to point at the sentence it concerns, so that the officer who opens it can go straight to the right record. Notification stores this in `related_record_id`, together with a `type` column that records what kind of record the id belongs to; because the same column can point at a sentence, an incident or a health record, the database cannot enforce it with an ordinary foreign key. This supports FR-027, the requirement that relevant staff be notified when a sentence nears completion.

Notification and Incident. The same `related_record_id` and `type` columns are used when a new disciplinary incident is logged, so that the notification sent to staff points directly to that incident, satisfying FR-028. The trade-off is the same as above: one column serves several purposes, so the database itself cannot prevent an id pointing at a record that no longer exists.

Notification and health records. The health module adds a third use of the same soft reference: when an appointment or follow-up is approaching, a notification's `related_record_id` points at the relevant `HealthRecord` or `MedicalAppointment`, with `type` set to `health`, satisfying FR-036's requirement to notify medical staff of upcoming appointments and follow-ups.

AuditLog and any entity. The audit log must describe changes to any part of the system, whether that is an inmate, a cell, a case or a health record, and it does this by writing down the name of the entity that was changed and the id of the specific record, in the columns `entity_name` and `record_id`. Because one `AuditLog` table serves every other entity, this pair acts as a flexible pointer rather than a real foreign key, which is deliberate: it means log entries stay valid even if the original record is later deleted, which is exactly what an audit trail should do, and it is what FR-025 requires when it asks that every create, update and delete action be logged.

Appendix A: Summary Table of All Relationships

The table below lists every relationship in this document in one place. The parent is the "one" side and the child is the "many" side, which holds the foreign key.

No.	Parent (one)	Child (many)	Foreign key
R1	Role	User	User.role_id -> Role.role_id
R2	Inmate	CellAssignment	CellAssignment.inmate_id -> Inmate.inmate_id
R3	Cell	CellAssignment	CellAssignment.cell_id -> Cell.cell_id
R4	User	CellAssignment	CellAssignment.assigned_by -> User.user_id
R5	Inmate	HealthRecord	HealthRecord.inmate_id -> Inmate.inmate_id
R6	User	HealthRecord	HealthRecord.created_by -> User.user_id
R7	HealthRecord	MedicalCondition	MedicalCondition.health_record_id -> HealthRecord.health_record_id
R8	User	MedicalCondition	MedicalCondition.recorded_by -> User.user_id
R9	HealthRecord	Consultation	Consultation.health_record_id -> HealthRecord.health_record_id
R10	User	Consultation	Consultation.consulted_by -> User.user_id
R11	Consultation	Medication	Medication.consultation_id -> Consultation.consultation_id
R12	User	Medication	Medication.prescribed_by -> User.user_id
R13	Medication	MedicationAdministration	MedicationAdministration.medication_id -> Medication.medication_id
R14	User	MedicationAdministration	MedicationAdministration.administered_by -> User.user_id
R15	HealthRecord	MedicalAppointment	MedicalAppointment.health_record_id -> HealthRecord.health_record_id
R16	User	MedicalAppointment	MedicalAppointment.scheduled_by -> User.user_id
R17	Inmate	Case	Case.inmate_id -> Inmate.inmate_id
R18	User	Case	Case.recorded_by -> User.user_id
R19	Case	CaseCrime	CaseCrime.case_id -> Case.case_id
R20	Crime	CaseCrime	CaseCrime.crime_id -> Crime.crime_id
R21	Case	Sentence	Sentence.case_id -> Case.case_id
R22	Inmate	Sentence	Sentence.inmate_id -> Inmate.inmate_id
R23	User	Incident	Incident.reported_by -> User.user_id
R24	Incident	IncidentInmate	IncidentInmate.incident_id -> Incident.incident_id
R25	Inmate	IncidentInmate	IncidentInmate.inmate_id -> Inmate.inmate_id
R26	Visitor	Visit	Visit.visitor_id -> Visitor.visitor_id
R27	Inmate	Visit	Visit.inmate_id -> Inmate.inmate_id
R28	User	Visit	Visit.logged_by -> User.user_id
R29	Inmate	Enrollment	Enrollment.inmate_id -> Inmate.inmate_id
R30	RehabProgram	Enrollment	Enrollment.program_id -> RehabProgram.program_id

No.	Parent (one)	Child (many)	Foreign key
R31	Enrollment	AttendanceRecord	AttendanceRecord.enrollment_id -> Enrollment.enrollment_id
R32	User	AttendanceRecord	AttendanceRecord.recorded_by -> User.user_id
R33	User	AuditLog	AuditLog.user_id -> User.user_id
R34	User	Notification	Notification.recipient_user_id -> User.user_id

Note: where User appears as the parent above (assigned_by, created_by, recorded_by, consulted_by, prescribed_by, administered_by, scheduled_by, reported_by, logged_by), this only records which staff member carried out that action. The record itself still belongs to the inmate, case, or other entity shown in its other relationship above — for example, a HealthRecord belongs to the inmate (R5); User only records who created it (R6).

No.	Relationship	How it is linked
S1	Notification and Sentence	Notification.related_record_id -> Sentence (type = sentence)
S2	Notification and Incident	Notification.related_record_id -> Incident (type = incident)
S3	Notification and health records	Notification.related_record_id -> HealthRecord / MedicalAppointment (type = health)
S4	AuditLog and any entity	AuditLog.entity_name + AuditLog.record_id -> any table